VeriVote-J — Architecture Document

Version: 1.0.1 (Phase 0 baseline; review fixes applied) **Date:** 2026-11-07 **Status:** Approved for Phase 0 **Project:** Cryptographically Verifiable Java E-Voting Platform

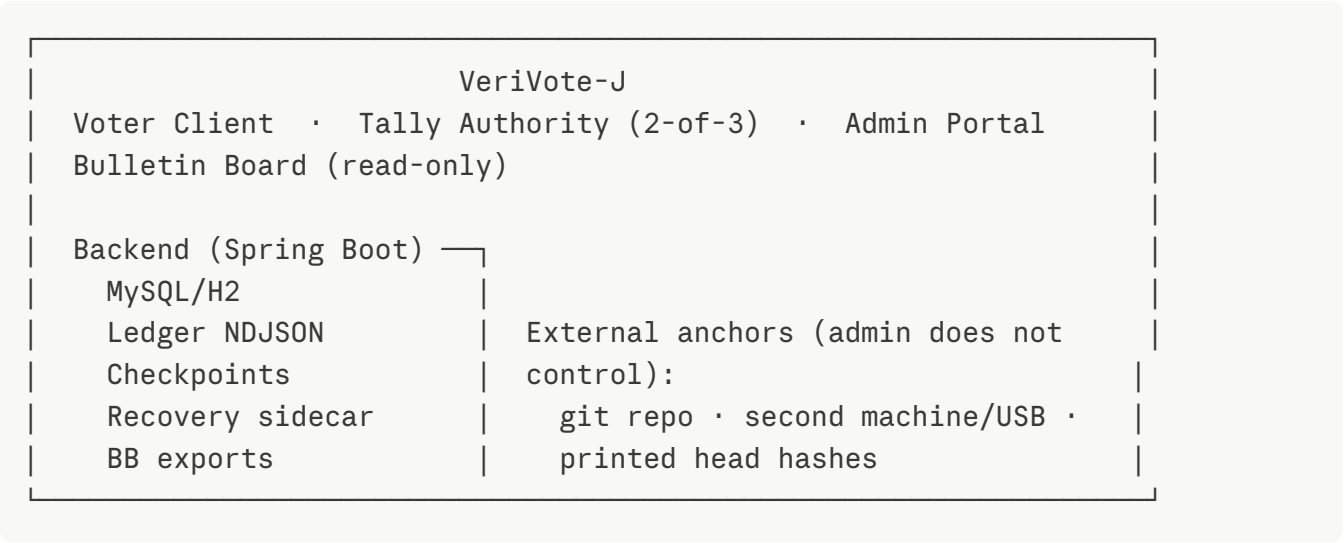
Table of Contents

1. Design Principles
2. System Context
3. Threat Model
4. Data Model
5. Ledger
6. Cryptography
7. Component Interfaces
8. Flow Diagrams
9. API Surface
10. Limitations
11. Phase Plan
12. Pre-coding Checklist
13. Remaining Code Fixes
14. Verdict

1. Design Principles

1. **Anonymity by separation** — no FK path from `voter` to `vote`. `spent_token` is the only bridge and carries no identity.
2. **Tamper-evidence over tamper-prevention** — the ledger is truth anchor for *what votes exist and their content*; the DB is a queryable cache. Tally refuses to run unless DB reconciles to ledger.
3. **Verify in ≤3 clicks after login.**
4. **Honest scope** — no blockchain, no consensus, no biometrics. Every limitation is documented.
5. **Every feature maps to a threat-model row.** Features without a row are cut.

2. System Context



Deployment: single machine for demo. External anchors on separate media. Tally Authority as separate JVM with own keystore. Shamir 2-of-3: Tally Authority · Election Commission · Observer.

Tech stack:

Layer	Tech
Frontend	JavaFX (Scene Builder, FontAwesomeFX)
Backend	Spring Boot (REST)
Database	H2 (dev) / MySQL (demo)
Crypto	`java.security`, `javax.crypto`, BouncyCastle (lightweight API)
Ledger	Append-only NDJSON files + DB index
2FA	TOTP (RFC 6238)
Bulletin board	Read-only REST + static JSON export
External sinks	Local dir + git repo + terminal print

3. Threat Model

#	Asset	Adversary	Attack	Mitigation	Status
T1	Ballot secrecy	Curious admin	Read vote table	No FK voter→vote; identity severed at issuance	Mitigated
T2	Ballot secrecy	Network	Sniff vote	RSA-OAEP client-side encryption	Mitigated
T3	Vote integrity	DB admin	UPDATE vote row	Hash-chained ledger; per-block hash check	Mitigated
T4	Vote integrity	DB admin	DELETE vote / truncate chain	Signed checkpoints; external sinks	Mitigated
T5	Double voting	Voter	Vote twice	`spent_token` PK; `has_voted` at issuance	Mitigated
T6	Ballot stuffing	Admin	Fabricated votes	Valid blind sig required; issuance reconciliation; admin still holds signing key (L15)	Partial → L15
T7	Coercion	Coercer	Forced vote	**Not mitigated** → L1	Future
T8	Verification failure	Lazy voter	No verification	≤3 clicks; auto-link	Mitigated
T9	Tally integrity	Tally authority	Wrong count	Publishes counts + chain root + shuffled ballots; no ZK → L7	Partial
T10	Credential theft	Network	Password steal	TOTP 2FA	Mitigated
T11	Ballot secrecy	Admin w/logs	Correlate order+timestamps	Shuffle; minute timestamps; vote/spent_token have no timestamps	Mitigated
T12	Chain integrity	Admin w/server	Rewrite chain + re-sign	External anchors	Mitigated if anchors used
T13	Client integrity	Server compromise	Lies `included:true`	Client verifies Merkle proof + checkpoint sig (pinned)	Mitigated
T14	Ballot secrecy	Network observer	IP/timing/TLS correlation	Client random delay 30–300s; IP → L4b	Partial
T15	Ballot integrity	Admin	Flip `has_voted`	Published issuance counter; regression detection	Partial (delayed)

#	Asset	Adversary	Attack	Mitigation	Status
T16	Tally correctness	Dishonest tally	Incorrect shuffle	**Not mitigated** → L7	Future
T17	Cross-election replay	Network	Replay sig	Domain-separated hash + per-election key	Mitigated
T18	Token as identifier	Logging	Sig bytes logged	Non-deterministic sigs; dedupe on `token_hash`	Mitigated
T19	Ledger integrity	Replay spammer	Fill ledger with orphans	`spent_token` before ledger; rate limit	Mitigated
T20	Vote integrity	Concurrent attacker	Race same token	`SELECT FOR UPDATE` + `spent_token` PK	Mitigated
T21	Audit integrity	DB admin	Delete vote, hide behind marker	In-chain ORPHAN markers; voteHash↔DB reconciliation	Mitigated
T22	Ledger integrity	Crash	Orphan after rollback	In-chain marker via `afterCompletion`; fallback log	Mitigated
T23	Timing correlation	Admin	`vote.created_at`	Timestamp columns dropped	Mitigated
T24	Ambiguous missing row	Admin vs. crash	Block with no row, no marker	ALERT + logged recovery + ciphertext-in-block rebuild	Partial
T25	Vote deletion	Admin	Delete row, accept orphan	Precondition (no spent_token OR retry) + two operator sigs + published count	Partial
T26	Ballot linkage	Admin w/public data	Match ballotId/receipt to token	Publish ballotId, never receipt nonce	Mitigated
T27	Post-CLOSE tampering	Admin	Append blocks after close	`forceNow()` at CLOSED anchors `closeHeadHash`	Mitigated

4. Data Model

4.1 Schema

```
CREATE TABLE voter (
  id               BIGINT PRIMARY KEY AUTO_INCREMENT,
  national_id_hmac VARCHAR(128) UNIQUE NOT NULL,    -- HMAC-SHA-256, keystore
key
  display_name     VARCHAR(255) NOT NULL,
```

```
  password_hash    VARCHAR(255) NOT NULL,           -- BCrypt
  totp_secret      VARBINARY(256),
  has_voted        BOOLEAN DEFAULT FALSE,           -- flipped at ISSUANCE
  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE spent_token (
  token_hash  VARCHAR(64) PRIMARY KEY,              -- SHA-256(SPEND-
v1||eid||token)
  election_id BIGINT NOT NULL
  -- NO timestamps. NO voter_id. Anonymity boundary.
);

CREATE TABLE vote (
  id             BIGINT PRIMARY KEY AUTO_INCREMENT,
  election_id    BIGINT NOT NULL,
  token_hash     VARCHAR(64) NOT NULL,
  receipt_hash   VARCHAR(64) NOT NULL UNIQUE,
  encrypted_vote TEXT NOT NULL,                     -- base64 of 256-byte
RSA-OAEP ciphertext
  block_height   BIGINT NOT NULL,
  block_hash     VARCHAR(64) NOT NULL
  -- NO client_nonce. NO timestamps. NO voter_id.
);

CREATE TABLE checkpoint (
  id           BIGINT PRIMARY KEY AUTO_INCREMENT,
  tree_size    INT NOT NULL,
  merkle_root  VARCHAR(64) NOT NULL,
  head_hash    VARCHAR(64) NOT NULL,
  signature    TEXT NOT NULL,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE operator (
  id           BIGINT PRIMARY KEY AUTO_INCREMENT,
  name         VARCHAR(128) NOT NULL UNIQUE,
  ed25519_pub  VARCHAR(128) NOT NULL,               -- base64 X.509 SPKI
  role         ENUM('ADMIN','AUDIT','TALLY') NOT NULL,
  active_from  TIMESTAMP NOT NULL,
  active_to    TIMESTAMP NULL,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  created_by   VARCHAR(128) NOT NULL
);

CREATE TABLE recovery_event (
```

```
  id           BIGINT PRIMARY KEY AUTO_INCREMENT,
  block_height BIGINT NOT NULL,
  action       ENUM('ACCEPT_AS_ORPHAN','RECOVER_FROM_LEDGER') NOT NULL,
  block_hash   VARCHAR(64) NOT NULL,
  token_hash   VARCHAR(64) NOT NULL,
  reason       VARCHAR(500) NOT NULL,
  operator1    VARCHAR(128) NOT NULL,
  signature1   TEXT NOT NULL,
  operator2    VARCHAR(128) NULL,
  signature2   TEXT NULL,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uk_recovery_block (block_height)       -- not (height, action)
);

CREATE TABLE issuance_counter (
  election_id BIGINT PRIMARY KEY,
  issued      BIGINT NOT NULL DEFAULT 0,
  resets      BIGINT NOT NULL DEFAULT 0,
  FOREIGN KEY (election_id) REFERENCES election(id)
);

CREATE TABLE issuance_published (
  id           BIGINT PRIMARY KEY AUTO_INCREMENT,
  election_id  BIGINT NOT NULL,
  issued       BIGINT NOT NULL,
  resets       BIGINT NOT NULL,
  votes_cast   BIGINT NOT NULL,
  published_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  signature    TEXT NOT NULL,
  FOREIGN KEY (election_id) REFERENCES election(id)
);

CREATE TABLE issuance_reset (
  id            BIGINT PRIMARY KEY AUTO_INCREMENT,
  voter_id_hmac VARCHAR(64) NOT NULL,
  reset_by      VARCHAR(128) NOT NULL,
  reason        TEXT NOT NULL,
  signature     TEXT NOT NULL,
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE election (
  id       BIGINT PRIMARY KEY AUTO_INCREMENT,
  title    VARCHAR(255) NOT NULL,
  status   ENUM('SETUP','OPEN','CLOSED','TALLIED') DEFAULT 'SETUP',
  opens_at TIMESTAMP, closes_at TIMESTAMP
```

```
);

CREATE TABLE candidate (
  id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  election_id BIGINT NOT NULL,
  name        VARCHAR(255) NOT NULL,
  party       VARCHAR(255),
  FOREIGN KEY (election_id) REFERENCES election(id)
);
```

4.2 Anonymity invariant

Grep the schema for any FK path `voter.*→.*vote`. **Must be none.** `spent_token` is the only bridge and carries no identity, no timestamps. `vote` carries no `voter_id`, no `client_nonce`, no timestamps.

5. Ledger

5.1 Block record

```
public record Block(
    long height, String prevHash, String type, long refHeight,
    String voteHash, String tokenHash, String receiptHash,
    String ciphertextB64, long timestampSec, String merkleRoot, String
blockHash
) {
    public static final String VOTE = "VOTE", ORPHAN = "ORPHAN";
    public static final String ZERO = "0".repeat(64);
    public static final int CT_LEN = 256;
}
```

5.2 Block hash (fixed-width, domain-separated)

```
blockHash = SHA-256(
    "VERIVOTE-BLOCK-v1" ||
    typeByte(1=VOTE, 2=ORPHAN) ||
    u64be(height) ||
    u64be(refHeight) ||                    // VOTE: -1 as 0xFF..FF
    prevHash[32] ||
    voteHash[32] ||                        // ORPHAN: 32 zero bytes
    tokenHash[32] ||                       // ORPHAN: 32 zero bytes
    receiptHash[32] ||                     // ORPHAN: 32 zero bytes
    u64be(timestampSec) ||                 // epoch seconds, minute-rounded
    merkleRoot[32]
)
```

`ciphertextB64` is not in the hash. It is trusted only through `voteHash`. `load()`, `range()` read, and tally all verify `SHA-256(base64decode(ciphertextB64)) == voteHash`. A mismatch is fatal.

5.3 Merkle leaves (per-type)

Type	Leaf data
VOTE	`hexToBytes(voteHash)`
ORPHAN	`"VERIVOTE-ORPHAN-v1" || u64be(refHeight)`

Tree hashes: `leaf = SHA-256(0x00 || data)`, `node = SHA-256(0x01 || l || r)`, `EMPTY_ROOT = SHA-256(0x02)`.

5.4 Files

Path	Purpose
`ledger/blocks.ndjson`	Append-only blocks
`ledger/checkpoints/ckpt-{treeSize}.json`	Signed checkpoints
`ledger/checkpoints/sink.ndjson`	Sink index
`ledger/recovery.ndjson`	Recovery sidecar
`ledger/orphans.fallback.ndjson`	Fallback for `markOrphan` failure
`ledger/blocks.ndjson.torn`	Repaired tail

Path	Purpose
`ledger/repairs.log`	Repair history

5.5 Load-time rules

- **Trailing line without newline** that fails parse/hash → torn tail. Move bytes to `.torn`, truncate in place, resume. WARN in `repairs.log`.
- **Any newline-terminated line** that fails parse/hash → **fatal**, refuse to start. Treat as tampering.
- Any write error during runtime → ledger disabled (503). No silent divergence.

5.6 `LedgerService` interface

```
public interface LedgerService {
    record Appended(long height, String blockHash, String voteHash) {}
    Appended append(byte[] ciphertext, byte[] tokenHash, byte[] receiptHash);
    void markOrphan(long height);              // idempotent
    boolean isOrphaned(long height);
    long blockCount();                         // total chain length
    long realVoteCount();                      // VOTE blocks not orphaned
    long orphanCount();
    String headHash();
    Optional<Block> at(long height);
    List<Block> range(long fromIncl, long toExcl);
    byte[] rootAt(long treeSize);              // root over leaves
[0..treeSize)
    List<byte[]> proof(long height, long treeSize);
}
```

Single-writer, `synchronized`, `FileChannel.force(true)` before return on every append/markOrphan.

6. Cryptography

6.1 Blind signature

- BC lightweight API: `RSABlindingEngine` + `PSSSigner`. **Not RFC 9474-conforming** — report wording: "blind RSA-PSS (RFC 9474 family)".
- Message preimage: `"VERIVOTE-TOKEN-v1" || int64be(electionId) || tokenPlain`.
- Per-election key; `verify()` fails if wrong key.

- `unblind` left-pads to `modLen` (BC strips leading zeros ~1/256 of signatures).
- `modLen = (bitLength + 7) / 8`.
- Signatures are **probabilistic** (PSS salt). Dedupe only on `token_hash`.

6.2 Vote encryption

- RSA-OAEP-SHA-256, 2048-bit. Output 256 bytes.
- Plaintext: `{electionId, candidateId, ballotId}` where `ballotId = SecureRandom(32)`.
- **`clientNonce` travels only in `/vote` request** to compute `receiptHash`; never stored or encrypted.

6.3 Domain table

Purpose	Preimage
Token signature	`"VERIVOTE-TOKEN-v1" || int64be(eid) || tokenPlain`
Spend hash	`SHA-256("VERIVOTE-SPEND-v1" || int64be(eid) || tokenPlain)`
Receipt hash	`SHA-256("VERIVOTE-RECEIPT-v1" || int64be(eid) || nonce || tokenHash)`
Block hash	§5.2
Merkle leaf	`SHA-256(0x00 || data)`
Merkle node	`SHA-256(0x01 || l || r)`
Empty tree	`SHA-256(0x02)`
ORPHAN leaf	`leafHash("VERIVOTE-ORPHAN-v1" || u64be(refHeight))`
Checkpoint sig	`"VERIVOTE-CKPT-v1" || u32be(treeSize) || root || head`
Recovery sig	`"VERIVOTE-RECOVERY-v1" || u64be(height) || ascii(action) || 0x00 || blockHash || tokenHash || reason`
Issuance-published sig	`"VERIVOTE-ISSUANCE-v1" || u64be(eid) || u64be(issued) || u64be(resets) || u64be(votesCast) || u64be(publishedAtSec)`

6.4 Tally authority

- RSA-2048 vote-decryption key, **Shamir 2-of-3** at rest.
- At tally: reconstruct, decrypt, **best-effort zeroize** (`Arrays.fill` on PKCS#8; JVM retains parsed BigInteger copies — L3).
- **Shuffle before decrypt.** Not verifiable (L7).
- **Tally refuses unless audit is clean AND every ciphertext hashes to its VOTE-block leaf.**

7. Component Interfaces

7.1 `CheckpointService`

```
public interface CheckpointService {
    record Checkpoint(long treeSize, String merkleRoot, String headHash,
                      String signature, long createdAtSec) {}
    Optional<Checkpoint> forceNow();     // empty ledger → empty; unchanged
size → latest
    void maybeCheckpoint();              // ≥10 new blocks OR ≥60s with new
blocks
    Optional<Checkpoint> latest();
    List<Checkpoint> all();
}
```

- Snapshot-by-size: `size = blockCount()` , `head = at(size-1)` , `root = rootAt(size)` .
- Refuses to sign if chain diverged from last checkpoint or truncated.
- Sinks written **before** DB insert. If `< minExternalSinks` external sinks succeed → throw, store nothing.
- Timer polls every 5 s.
- `forceNow()` at CLOSED anchors `closeHeadHash` immediately (T27).

Sinks: `FileSink` (external), `GitSink` (external, `git add -- sink.ndjson ckpt-*.json` with `-c commit.gpgsign=false -c user.email=... -c user.name=...`), `PrintSink` (not external, demo only).

7.2 `RecoveryService`

```
public interface RecoveryService {
    enum Action { ACCEPT_AS_ORPHAN, RECOVER_FROM_LEDGER }
    record RecoveryRequest(long blockHeight, Action action, String reason,
                           String operator1, String operator2,
                           String signature1, String signature2) {}
    record RecoveryResult(long eventId, long blockHeight, Action action) {}
    byte[] signingPreimage(long blockHeight, Action action, String reason);
    RecoveryResult execute(RecoveryRequest req);
}

public class RecoveryException extends RuntimeException {
    public enum Code { BAD_REQUEST, BLOCK_NOT_VOTE, BLOCK_MISSING,
BLOCK_ALREADY_ORPHANED,
        SIGNATURE_INVALID, OPERATOR_UNKNOWN, OPERATOR_INACTIVE,
OPERATOR_ROLE_MISMATCH,
        SAME_OPERATOR, PRECONDITION_FAILED, ALREADY_RECOVERED,
ELECTION_NOT_CLOSED }
}
```

Preconditions:

Action	Precondition
Both	`election.status == CLOSED`; block at height is VOTE; `!ledger.isOrphaned(height)`; `!voteRowExistsAt(height)`
`ACCEPT_AS_ORPHAN`	Two distinct active AUDIT operators; **NOT** (`spent_token` exists AND no retry committed)
`RECOVER_FROM_LEDGER`	One active AUDIT operator; **`spent_token` MUST exist**; no retry committed

Side-effect order:

```
RecoveryResult result = tx.execute(st -> {
    validateSignatures();
    validatePreconditions();
    if (RECOVER) data.insertVote(...);          // base64 stored as-is
    long id = data.insertEvent(ev);             // unique key on block_height
    sidecar.append(ev);                         // fsync
    return new RecoveryResult(id, h, action);
});
if (action == ACCEPT_AS_ORPHAN) ledger.markOrphan(h);   // outside
transaction, idempotent
return result;
```

`markOrphan` is outside the transaction (irreversible). Retry-after-crash handled by **event-first** check (fix F1, §13).

7.3 `AuditService`

```
public record TamperReport(boolean clean, long auditedAtSec, long
ledgerBlockCount,
        long ledgerRealVoteCount, long ledgerOrphanCount, long dbVoteCount,
List<Finding> findings) {
    public boolean blocksTally() { return !clean; }
}
public record Finding(Severity severity, String code, String message,
                      Long blockHeight, Long treeSize, String remediation) {
    public enum Severity { RED, ALERT, WARN, INFO }
}
```

`clean = no RED and no ALERT`. WARN and INFO do not block tally.

Full findings table:

Code	Sev	Condition
`LEDGER_UNREADABLE`	RED	Cannot read ledger file
`CHAIN_HASH_MISMATCH`	RED	`computeHash() != blockHash`
`CHAIN_LINK_BROKEN`	RED	`prevHash != previous.blockHash`
`MALFORMED_LINE`	RED	JSON parse fails on newline-terminated line
`CIPHERTEXT_MISMATCH`	RED	`SHA-256(ct) != voteHash` on VOTE block
`MERKLE_ROOT_MISMATCH`	RED	Stored root ≠ computed root

Code	Sev	Condition
`ORPHAN_MARKER_WITHOUT_TARGET`	RED	ORPHAN references nonexistent/wrong-type VOTE
`DOUBLE_ORPHAN_MARKER`	RED	Two ORPHAN blocks reference same VOTE
`CHAIN_TRUNCATED`	RED	`ledger.blockCount() < sink.treeSize`
`SINK_DIVERGED`	RED	Sink checkpoint doesn't verify against chain at that size
`SINK_MALFORMED`	RED	Sink file line unparseable
`CHECKPOINT_BAD_SIGNATURE`	RED	Ed25519 verification fails
`DB_CIPHERTEXT_MISMATCH`	RED	DB `encrypted_vote` hash ≠ block's `voteHash`
`DB_FIELD_MISMATCH`	RED	token/receipt/block hash differs from ledger
`VOTE_ROW_WITHOUT_BLOCK`	RED	Row points at non-VOTE/nonexistent height
`ORPHAN_HAS_ROW`	RED	Row exists for orphaned block
`DUPLICATE_ROW`	RED	Two rows at same height
`DUPLICATE_TOKEN_HASH`	RED	**>1 non-orphan VOTE with same tokenHash has a matching row**
`VOTES_EXCEED_ISSUED`	RED	`realVoteCount > issued`
`RECOVERY_EVENT_UNSIGNED`	RED	Either signature fails, or `operator1==operator2` on ACCEPT
`ISSUANCE_REGRESSED`	RED	Any published counter decreased
`BLOCK_WITHOUT_VOTE_ROW`	ALERT	VOTE block has no row, no marker, no event
`NO_EXTERNAL_ANCHORS`	ALERT	Blocks exist, no external anchor
`SINK_AHEAD_OF_DB`	WARN	Sink checkpoint missing from DB, verifies
`DB_AHEAD_OF_SINK`	WARN	DB checkpoint missing from sink, verifies
`ISSUANCE_MISMATCH`	WARN	`issued != hasVoted + resets`
`SINK_UNREADABLE`	WARN	Sink I/O error
`TORN_TAIL_PRESENT`	WARN	Trailing bytes without newline
`TORN_TAIL_REPAIRED`	INFO	`.torn` file exists
`UNANCHORED_TAIL`	INFO	Blocks after last anchor

Duplicate finding suppression: dedupe by `(code, treeSize)`, merge source names into one finding.

Early return on fatal parse: if `parse()` returns `fatal != null`, emit exactly one finding and skip all later checks.

`DUPLICATE_TOKEN_HASH` rule:

```
Map<String, List<Block>> byTok = ...;         // non-orphan VOTE blocks by
tokenHash
Set<Long> rowHeights = ...;                   // DB vote row heights
for (var g : byTok.values()) {
    long committed = g.stream().filter(x ->
rowHeights.contains(x.height())).count();
    if (g.size() > 1 && committed > 1) → RED
    // committed <= 1: unmatched twins become BLOCK_WITHOUT_VOTE_ROW (retry
case)
}
```

`BLOCK_WITHOUT_VOTE_ROW` message classification:

```
if (spentTokenExists(b.tokenHash())) {
    boolean retry = byTok.getOrDefault(b.tokenHash(), List.of(b))
        .stream().anyMatch(x -> x.height() != b.height() &&
rowHeights.contains(x.height()));
    why = retry
        ? "a committed retry with the same tokenHash exists ->
ACCEPT_AS_ORPHAN (retry case)"
        : "spent_token EXISTS and no retry committed -> RECOVER_FROM_LEDGER
or investigate deletion";
} else why = "no spent_token -> crash before commit -> ACCEPT_AS_ORPHAN (two
operators)";
```

`ISSUANCE_REGRESSED`: compare current `issuance_counter` against `issuance_published` latest. Any decrement → RED. Published snapshot is written to `issuance_published` **and** to external sinks.

Audit reads ledger file from disk, not the in-memory `FileLedgerService.tree()`. Uses `LedgerFileParser.parse(file)` independently. `CHAIN_HASH_MISMATCH` per block enforces agreement.

8. Flow Diagrams

8.1 Issuance (authenticated) → Vote (anonymous)

```
Session A (auth): issuance
  1. Client: tokenPlain ← SecureRandom(32); m = "VERIVOTE-TOKEN-
v1"||int64be(eid)||tokenPlain
  2. r ← blinding; blinded = Blind(m, r)
  3. POST /token/request { eid, blinded }
  4. Server: verify session; voter.has_voted == FALSE
  5. atomic: has_voted = TRUE; issuance_counter.issued++
  6. blindSig = RSA_priv(blinded)             // no voter link stored
  7. return blindSig + issuanceReceipt
  8. Client: sig = Unblind(blindSig, r); tokenProof = (tokenPlain, sig)
     Session A ends.

Random delay: client sleeps uniform(30,300)s.

Session B (anonymous): voting
  9.  ballotId ← SecureRandom(32)
 10.  encryptedVote = RSA-OAEP(tallyPub, {eid, candidateId, ballotId})
 11.  clientNonce ← SecureRandom(32)                   // for receipt only
 12.  POST /vote { eid, tokenPlain, sig, encryptedVote, clientNonce }
 13.  Server: verify sig BEFORE lock
 14.  tokenHash = SHA-256(SPEND-v1||eid||tokenPlain)
 15.  receiptHash = SHA-256(RECEIPT-v1||eid||nonce||tokenHash)
 16.  @Transactional:
        SELECT election FOR UPDATE; status must be OPEN
        INSERT spent_token(tokenHash) saveAndFlush      // unique PK
        LedgerService.Appended a = ledger.append(ct, tokenHash, receiptHash)
// fsync
        register afterCompletion → markOrphan on STATUS_ROLLED_BACK
        INSERT vote(eid, tokenHash, receiptHash, base64(ct), a.height,
a.blockHash)
      COMMIT
 17.  return { receiptHash, a.height }
```

Anonymity properties: server never sees `tokenPlain` at issuance (blinded) and never sees a session at vote. No DB row links voter identity to `tokenHash`.

8.2 Receipt verification (client-side)

```
Client (pinned: tallyPub, ckptPub)
  1. GET /bb/verify/{receiptHash}
       → { status, index, treeSize, proof[], leafData }
  2. if status == NOT_FOUND → "pending, retry"
     if status == PENDING_INCLUSION → "pending checkpoint, auto-retry every
10s up to 5min"
  3. voteHash_local = SHA-256(encryptedVote)                // client's own
ciphertext
  4. assert leafData == voteHash_local              // cross-check
  5. GET /bb/checkpoints → find smallest treeSize' > index
  6. assert Ed25519.verify(ckptPub,
       "VERIVOTE-CKPT-v1"||u32(treeSize')||root||head, sig)
  7. assert MerkleVerifier.verify(voteHash_local, index, treeSize', proof,
root)
  8. ✅ or ❌
```

`index` and `treeSize'` come from the **signed checkpoint**, not the server's proof response.

8.3 Tally

```
Admin clicks "Close & Tally":
  1. election.status → CLOSED (SELECT FOR UPDATE)
  2. checkpoints.forceNow()                    // anchor closeHeadHash
immediately (T27)
  3. audit = auditService.audit()
     if audit.blocksTally() → TallyPreconditionException; election stays
CLOSED (re-runnable)
  4. Collect non-orphan VOTE blocks from ledger
     For each: verifyCiphertext(); assert unique tokenHash
  5. Shamir reconstruct tally key (2 of 3 shares)
  6. Fisher–Yates shuffle (SecureRandom)
  7. Decrypt each; extract {candidateId, ballotId}. A ciphertext that fails
to decrypt, or whose electionId/candidateId is invalid, is counted as INVALID
(no candidate credit, count published)
  8. Count per candidate
  9. Zeroize best-effort (L3)
 10. Publish /bb/tally:
       counts (incl. invalid), closeBlockHeight, closeHeadHash,
chainMerkleRoot,
       ballots: [{candidateId, ballotId}]
 11. election.status → TALLIED
```

Tally reads from ledger, not DB. Tally aborts leave election CLOSED — re-runnable.

8.4 Tamper detection demo

```
Admin → AuditService.audit() → TamperReport
  1. Re-read blocks.ndjson from disk; verify every hash + link
  2. Compare ledger head against all sink checkpoints
  3. Reconcile every VOTE block ↔ DB row (voteHash, tokenHash, receiptHash,
blockHash)
  4. Classify missing rows: ALERT / expected / RED
  5. Verify every recovery_event signature
  6. Compare issuance_counters against last published
Result: clean, alerts, warns, reds
```

Demo script:

1. `UPDATE vote SET encrypted_vote=... WHERE id=7;` → **RED** (`DB_CIPHERTEXT_MISMATCH`).
2. `DELETE FROM vote WHERE id=5;` → **ALERT** (`BLOCK_WITHOUT_VOTE_ROW`). Operator runs `RECOVER_FROM_LEDGER`, vote restored, audit clean.
3. Truncate `blocks.ndjson` → restart refuses (mid-file fatal) OR audit reports `CHAIN_TRUNCATED` vs sink.
4. Append garbage line to `blocks.ndjson` → restart refuses (newline-terminated fatal).

9. API Surface

Method	Path	Auth	Purpose
POST	`/api/register`	none	Create voter, return TOTP secret
POST	`/api/login`	none	Password → challenge
POST	`/api/login/totp`	challenge	TOTP → session
POST	`/api/token/request`	session	Blind-signed token (sets `has_voted`)
POST	`/api/vote`	**none**	Cast vote (anonymous by design)
GET	`/api/receipt/{hash}`	none	Receipt metadata
GET	`/api/election/current`	session	Election + candidates
GET	`/bb/receipts`	none	All receipt hashes
GET	`/bb/verify/{receiptHash}`	none	Inclusion proof
GET	`/bb/checkpoints`	none	Signed checkpoint history

Method	Path	Auth	Purpose
GET	`/bb/tally`	none	Counts + shuffled `[{candidateId, ballotId}]`
GET	`/bb/issuance`	none	`issued`, `resets`, `votesCast`, recovery count
POST	`/admin/election`	admin	Create election
POST	`/admin/election/{id}/open`	admin	Open
POST	`/admin/election/{id}/close`	admin	Close + force checkpoint + tally
POST	`/admin/reset-issuance`	admin	Logged `has_voted` reset
POST	`/admin/recover`	admin	Recovery execution
GET	`/admin/audit`	admin	Returns `TamperReport`
GET	`/admin/monitor`	admin	Live stats

Rate limiting: attempts-only, per source IP, 10/min, via `HandlerInterceptor` before transaction start. Never outcome-based.

Deleted: `/bb/find/{ballotId}` — reintroduced T11 linkage. Voters find their ballot by scanning the published list locally.

10. Limitations

> **L1 — Coercion and vote-selling not prevented.** Receipts allow ballot-box coercion. Additionally, because `ballotId` is published alongside the candidate, a voter can **sell their vote** by transferring `ballotId` after tally — a **transferable proof of choice**. This is a deliberate design trade-off: publishing `{candidateId, ballotId}` enables anyone to recount, which is the point of the bulletin board. Preventing both requires a coercion-resistant protocol (JCJ/Civitas), future work.

> **L2 — Single-machine tamper defense.** Ledger + checkpoint key + DB on one machine. Tamper-evidence relies on external anchors (second machine, git, printed head hashes). Production: HSM + offline signing ceremony.

> **L3 — Tally key distribution (partial).** Shamir 2-of-3 protects against single-share theft and single-trustee coercion at rest, but **not** against a malicious tally operator at tally time (2 shares reconstruct the full key). **Zeroization is best-effort only**: JCA's `RSAPrivateCrtKey` holds immutable `BigInteger`s that cannot be wiped; `Arrays.fill` on the PKCS#8 bytes clears the encoding but not the parsed copy, and the JVM may retain additional copies. Known JVM limitation.

> **L4 — Client-side correlation.** Compromised client can correlate issuance and vote sessions.

> **L4b — Network-layer correlation.** Server sees source IP at issuance and vote. Client random delay (30–300 s) mitigates timing but not IP/TLS fingerprint. With a small electorate the anonymity set is only the voters who issue in the same window, so timing correlation is easier. Production: Tor or mixnet.

> **L5 — No universal verifiability of tally.** Counts, pre-shuffle chain root, and shuffled decrypted ballots are published (recount-able), but no ZK proof of correct decryption. Future work: Chaum–Pedersen or Bayer–Groth.

> **L6 — Trusted setup.** Keys generated at first run; no DKG. Shamir split applied after generation. Client pinned keys embedded via post-bootstrap build step. Production: DKG + threshold decryption.

> **L7 — No verifiable shuffle.** Honest-but-curious only. Incorrect shuffle is undetectable. Future work: Bayer–Groth verifiable shuffle.

> **L8 — Single-writer assumption.** Ledger writer must run as one process. No leader election.

> **L9 — Ciphertext-in-ledger scales to demo only.** Each VOTE block stores ~344 bytes base64. Production: archival service with independent integrity proofs.

> **L10 — Single-election deployment.** `voter.has_voted` is global; ledger is one chain per election. Multi-election requires `voter_election(voter_id, election_id)` and per-election ledgers.

> **L11 — Ledger repair is operator-mediated.** Torn-tail repairs automatic and logged; newline-terminated corruption is fatal and requires operator intervention. Admin with root access can delete chain + torn file; external anchors catch this via manual comparison.

> **L12 — Reset hole.** Admin can reset `has_voted` for a voter whose token wasn't actually lost, letting them issue two valid tokens if the first is spent later. Published `resets` count caps damage and makes anomaly visible. Mitigation requires bind-to-identity checks that conflict with anonymity.

> **L13 — Operator registry is admin-managed.** Operators enrolled by election administrator before `OPEN`. Malicious admin can enroll themselves under multiple names to satisfy two-operator requirement. Published `recovery_event` rows include operator names, so abuse is visible after the fact but not preventable.

> **L14 — Recovery requires election CLOSED.** Recovery is only permitted after `CLOSED`. This prevents racing the tally. Retry-after-crash is supported via event-first idempotency.

> **L15 — Issuance trust gap.** The blind-signing key and `issuance_counter` live on the server the admin controls. A malicious admin can mint extra valid tokens, or issue to registered voters who never vote, and cast ballots with them. Published `issued` / `votesCast` / `resets` counts (T15) and `VOTES_EXCEED_ISSUED` expose this only when totals exceed what was issued, and cannot prove who actually voted. Production: split the signing key across operators or an HSM, publish the registered-voter count, and audit issuance against an independent electoral roll.

11. Phase Plan

Phase	Deliverable
0	Threat model doc; multi-module Maven skeleton
1	Schema + JPA entities; `crypto` package skeleton
2	`BlindSignatureService`, `MerkleTree` / `MerkleVerifier`, `Sha256`, `Hmac`, `Shamir`

Phase	Deliverable
3	Auth + TOTP
4	Token service + issuance counter
5	**`LedgerService`** + `LedgerFileParser` + tests
6	`VoteService` + `/vote` + concurrent double-spend test
7	`CheckpointService` + sinks + tests
8	**`AuditService`** + all findings + tests
9	**`RecoveryService`** + operator registry + sidecar + tests
10	Bulletin Board + `/bb/issuance` + issuance_published + regression detection
11	**`TallyService`** — audit precondition, forceNow, Shamir, shuffle, decrypt-from-ledger, publish
12	JavaFX client — four-state verify UI, local Merkle proof
13	JavaFX admin — audit, recovery, tally
14	Demo script + report

Critical path: 5 → 6 → 7 → 8 → 9 → 11. Everything else parallelizes.

Delivery order: to cut scope risk, build a thin vertical slice first (login → issue → vote → ledger → verify → tally, with audit stubbed), then layer on checkpoints, `AuditService` and `RecoveryService`.

12. Pre-coding Checklist

```
Crypto
□ Block hash: "VERIVOTE-BLOCK-v1" || fixed-width fields || merkleRoot
□ Checkpoint sig: "VERIVOTE-CKPT-v1" || u32(treeSize) || root || head
□ Recovery sig: "VERIVOTE-RECOVERY-v1" || u64(h) || action || 0x00 ||
blockHash || tokenHash || reason
□ Issuance-published sig: "VERIVOTE-ISSUANCE-v1" || u64(eid) || u64(issued)
|| u64(resets) || u64(votesCast) || u64(ts)
□ Blind sig: BC RSABlindingEngine + PSSSigner; unblind left-pads to modLen
□ Merkle: leaf H(0x00||data); node H(0x01||l||r); EMPTY_ROOT=H(0x02)
```

```
□ MerkleVerifier: spec-literal fn!=0 guard; proof.size() bound by ceil(log2
n)
□ rootAt(treeSize); rootAt(0)==EMPTY_ROOT;
rootAt(blockCount)==head.merkleRoot

Anonymity
□ No FK path voter→vote (grep schema)
□ /vote unauthenticated, no session, no cookie
□ has_voted flipped at issuance; issuance_counter.issued++
□ spent_token no voter_id, no timestamps
□ vote: no voter_id, no client_nonce, no timestamps; block_height NOT NULL,
block_hash NOT NULL
□ Vote plaintext = {electionId, candidateId, ballotId}; ballotId =
SecureRandom(32)
□ receiptHash = H(RECEIPT-v1||eid||nonce||tokenHash); nonce never stored,
never published
□ /bb/tally publishes shuffled [{candidateId, ballotId}]; /bb/find deleted
□ tokenHash = H(SPEND-v1||eid||tokenPlain)
□ Client random delay 30–300 s before /vote

Ledger
□ Block record: height, type, refHeight, prevHash, voteHash, tokenHash,
receiptHash,
   ciphertextB64, timestampSec, merkleRoot, blockHash
□ append(ciphertext, tokenHash, receiptHash) → ledger computes voteHash
□ Single writer; FileChannel.force(true) before return
□ ORPHAN: voteHash/tokenHash/receiptHash = ZERO; ciphertextB64 null;
refHeight = orphaned height
□ VOTE: refHeight = -1; ciphertextB64 present
□ Invariant: SHA-256(base64decode(ciphertextB64)) == voteHash, checked on
load/read/audit/tally
□ Load: newline-terminated bad line → fatal; trailing no-newline bad line →
torn + repair
□ markOrphan idempotent; outside recovery transaction; non-throwing in
afterCompletion
□ LedgerFileParser shared by FileLedgerService.load() and
AuditService.audit()

Vote endpoint
□ Signature verify before SELECT FOR UPDATE
□ SELECT FOR UPDATE on election (MySQL integration test)
□ spent_token saveAndFlush before ledger append
□ register afterCompletion BEFORE vote insert
□ Vote insert includes block_height and block_hash (single statement)
□ afterCompletion marks orphan only on STATUS_ROLLED_BACK
□ Concurrent double-spend: 1 success, 19×409, 1 spent_token, 1 VOTE block, 0
```

```
orphans
□ Crash-after-append: retry succeeds; audit reconciles

Checkpoint
□ Checkpoint signs (treeSize, root, head) with domain prefix
□ forceNow returns Optional; empty ledger → empty; unchanged size → latest
(idempotent, no re-publish)
□ Snapshot-by-size (root and head derived from same size)
□ Refuse to sign truncated or diverged chain
□ Sinks written BEFORE DB insert; minExternalSinks enforced
□ forceNow() called at CLOSED before tally publishes closeHeadHash
□ Checkpoints to external sinks: file + git + print; audit compares against
ALL

Audit
□ All findings codes present (§7.3)
□ clean = no RED and no ALERT; blocksTally() = !clean
□ DUPLICATE_TOKEN_HASH: only when >1 non-orphan VOTE with same tokenHash has
a row
□ BLOCK_WITHOUT_VOTE_ROW message classifies retry vs committed-deletion vs
crash
□ Early return on fatal parse
□ Dedupe checkpoint findings by (code, treeSize), merge source names
□ ISSUANCE_REGRESSED vs issuance_published
□ Re-reads ledger from disk

Recovery
□ ELECTION_NOT_CLOSED precondition
□ ACCEPT_AS_ORPHAN: 2 distinct active AUDIT operators, precondition (no spent
OR retry)
□ RECOVER_FROM_LEDGER: 1 operator, spent_token MUST exist, no retry
□ block_height unique key (not (height, action))
□ markOrphan outside transaction
□ Event-first retry (fix F1)
□ Base64 stored as-is in DB
□ Sidecar fsync

Tally
□ forceNow at CLOSED before audit
□ Audit precondition (blocksTally)
□ Non-orphan VOTE only
□ Verify ciphertext hash, unique tokenHash
□ Shamir 2-of-3 reconstruction
□ Shuffle before decrypt
□ Decrypt from ledger, not DB
□ Zeroize best-effort
```

```
□ Publish counts + closeHeadHash + chainMerkleRoot + shuffled [{candidateId,
ballotId}]
□ Abort leaves election CLOSED
□ Undecryptable or invalid ballots counted as INVALID and the count published

Docs
□ L1–L15 in limitations
□ T1–T27 in threat model
□ "≤3 clicks after login" consistent
□ "blind RSA-PSS (RFC 9474 family)", never "implements RFC 9474"
□ "one election per ledger" stated
□ "DB is a cache; ledger is truth anchor" backed by tally-from-ledger
```

13. Remaining Code Fixes

Fix F1 — recovery retry-after-crash. `execute()` must handle the case where a previous call committed `recovery_event` + sidecar but crashed before `markOrphan`:

```
public RecoveryResult execute(RecoveryRequest r) {
    validateRequestShape(r);   // BAD_REQUEST checks
    // 1. Event-first: is there already a recovery_event at this height?
    var existing = data.eventAt(r.blockHeight());
    if (existing.isPresent()) {
        var ev = existing.get();
        if (ev.action().equals(r.action().name())
                && r.action() == Action.ACCEPT_AS_ORPHAN
                && !ledger.isOrphaned(r.blockHeight())) {
            // Resume: signatures already validated when event was written.
            ledger.markOrphan(r.blockHeight());
            return new RecoveryResult(ev.id(), r.blockHeight(), r.action());
        }
        throw new RecoveryException(ALREADY_RECOVERED, "height " +
r.blockHeight());
    }
    // 2. Normal path with full validation
    RecoveryResult res = tx.execute(st -> {
        validateSignatures(r);
        validatePreconditions(r);
        if (r.action() == Action.RECOVER_FROM_LEDGER) data.insertVote(...);
        long id = data.insertEvent(ev);      // unique key on block_height
        sidecar.append(ev);
        return new RecoveryResult(id, r.blockHeight(), r.action());
    });
```

```
    if (r.action() == Action.ACCEPT_AS_ORPHAN)
ledger.markOrphan(r.blockHeight());
    return res;
}
```

The event-first check makes recovery idempotent at the API level without losing the operator signature audit trail.

Fix F2 — `FakeRecoveryData.insertEvent` must enforce the unique key. Without it, `racingAccepts_exactlyOneWins` is a false positive:

```
@Override public synchronized long insertEvent(RecoveryData.EventRow e) {
    if (events.stream().anyMatch(x -> x.blockHeight() == e.blockHeight()))
        throw new DuplicateEventException();
    events.add(e);
    return events.size();
}
```

14. Verdict

Architecture is internally consistent, every review round applied, all limitations honest, all threat-model rows mapped to concrete mitigations and statuses. The three money-shots (tamper detection, recovery, receipt verification) all have executable tests and deterministic demo scripts.

Ready for Phase 0. The two remaining implementation artifacts are `TallyService` and the client verifier, both of which depend only on services already specified here.

Open before coding: fixes F1 and F2 (§13); `TallyService` and the client verifier are still to be written. This revision corrected receipt-verification step 4, added invalid-ballot handling to the tally (§8.3), downgraded T6 to Partial, and added L15.

End of document.